Editor web de partidas Pokémon

Especificación del proyecto para desarrollar con Claude Code · v0.1 · noviembre 2026

1. Resumen

Web donde cualquier persona sube el archivo de guardado (**.sav**) de cualquier juego de Pokémon, lo ve y edita desde el navegador (también en el móvil) y lo descarga de nuevo listo para usar. Sobre esa base se añaden herramientas de ayuda: completar la Pokédex, encontrar objetos y MTs, y mapas interactivos personalizados según la partida del usuario.

Lo mínimo que tiene que existir: subir .sav → ver y editar → descargar .sav válido.

2. Principios del proyecto

- **Todo se procesa en el navegador.** La partida nunca se sube a un servidor: más privacidad, coste de hosting casi cero y funciona offline una vez cargada.
- **Nunca destruir la partida original.** Antes de exportar se ofrece descargar una copia del archivo original sin tocar.
- **Código abierto (GPL-3.0).** Se usa PKHeX.Core, que es GPL, así que el proyecto entero se publica con esa licencia. Los anuncios están permitidos.
- **Sin ROMs ni contenido de Nintendo en el repositorio.** Ni ROMs, ni partidas reales de terceros. Cuidado con sprites y artes oficiales (ver Riesgos).
- **Móvil primero.** Mucha gente juega en emulador de móvil; la web tiene que ser cómoda en pantalla pequeña.

3. Stack técnico propuesto

La idea clave es no reimplementar los formatos de guardado: **PKHeX.Core** (librería C# de PKHeX, disponible en NuGet) ya lee y escribe partidas de todas las generaciones, con checksums, cifrado y comprobación de legalidad. Se ejecuta en el navegador compilándola a WebAssembly con .NET.

Pieza	Propuesta	Notas
Lógica de partidas	PKHeX.Core (NuGet) en .NET 8/9 compilado a WebAssembly	Validar en la Fase 0 que funciona en WASM y cuánto pesa.
Puente con la UI	Opción A: Blazor WebAssembly (todo en C#). Opción B: .NET WASM con [JSExport] + frontend en TypeScript	Decidir tras la Fase 0. La B aprovecha mejor la experiencia web (JS/TS) del desarrollador.
Frontend	TypeScript + Vite (React o Svelte) si se elige la opción B	UI responsive, modo oscuro.
Mapas	Leaflet con CRS.Simple (mapas de imagen, no geográficos)	Teselas generadas a partir de imágenes de los mapas.
Datos de juego	PokéAPI (encuentros por versión) + JSON propios generados con scripts	Los datos se precalculan y se sirven como JSON estático.
Hosting	Estático: Cloudflare Pages, Netlify o GitHub Pages	Sin backend en el MVP.
Tests	xUnit para el lado .NET, Vitest/Playwright para la UI	Partidas de prueba generadas, no partidas reales.

4. Fases

Fase 0 · Prueba técnica (spike)

- Crear un proyecto mínimo que cargue PKHeX.Core en el navegador.
- Subir un .sav, detectar automáticamente el juego y la generación, y mostrar nombre del entrenador y equipo.
- Medir el peso de la descarga inicial y el tiempo de carga; probar trimming y compresión.
- **Decidir** entre Blazor y JSExport + TypeScript y documentar la decisión.

Fase 1 · MVP del editor

- Carga por arrastrar y soltar o selector de archivo; detección del juego con mensaje claro si no es compatible.
- Vista de entrenador (nombre, ID, dinero, medallas, tiempo de juego), equipo y cajas del PC.
- Editar Pokémon: especie, nivel, mote, naturaleza, habilidad, movimientos, IVs, EVs, shiny, objeto.
- Editar dinero y nombre del entrenador.
- Exportar el .sav con checksums correctos y descargar copia del original.
- Avisos de legalidad básicos usando el comprobador de PKHeX.Core.

Fase 2 · Edición ampliada

- Mochila completa (objetos, MTs, bayas, objetos clave) con buscador.
- Pokédex: marcar vistos y capturados.
- Importar y exportar Pokémon sueltos (.pk*) y formato de texto de Showdown.
- Deshacer y rehacer cambios.

Fase 3 · Asistente de Pokédex (empezar por Blanco/Negro)

- Leer de la partida qué Pokémon faltan y mostrar dónde se encuentran en ese juego concreto: ruta, método (hierba, surf, pesca…), estación, nivel y probabilidad.
- Indicar los que solo se consiguen por evolución, intercambio, evento o en la versión contraria.
- Filtros: por ruta, por método, solo los disponibles según el progreso (medallas).

Fase 4 · Buscador de objetos y MTs

- Buscar cualquier objeto o MT y ver si ya lo tienes y dónde conseguirlo (suelo, tienda, NPC, recompensa).
- Usar los flags de evento de la partida para saber qué objetos del mapa ya se han recogido.

Fase 5 · Mapas interactivos

- Mapa de la región con zoom; capas de Pokémon, objetos, entrenadores y puntos de interés.
- Modo personal: con la partida cargada, solo se muestra lo que al jugador le falta.
- Empezar con un juego y ampliar progresivamente.

Fase 6 · Extras

- Comparar dos partidas (qué tiene una que no tenga la otra) para preparar intercambios.
- Analizador de equipo competitivo (debilidades, cobertura, stats).
- Compartir el equipo como imagen o enlace (sin subir la partida).
- Integración futura con una app homebrew de 3DS que envíe la partida a la web.

5. Datos externos

- **Encuentros salvajes**: PokéAPI tiene encuentros por ubicación y versión. Comprobar cobertura juego a juego; completar con datos propios si faltan.

- **Objetos en el mapa, entrenadores y flags de evento**: no están en PokéAPI. Se obtienen de las decompilaciones públicas (por ejemplo aletheia para Blanco/Negro, pret para otras generaciones) o extrayéndolos con scripts de una ROM propia. En el repositorio solo se guardan los JSON generados, nunca la ROM.
- **Imágenes de mapas**: generarlas a partir de datos propios o capturas propias; revisar la licencia de cualquier recurso de terceros antes de usarlo.
- Todos los datos se generan con scripts reproducibles en una carpeta `tools/` y se versionan como JSON.

6. Requisitos no funcionales

- Funciona en Chrome, Firefox y Safari, en escritorio y móvil.
- Primera carga razonable: mostrar la interfaz enseguida y cargar el motor WASM en segundo plano con indicador de progreso.
- Español e inglés desde el principio (i18n), ampliable.
- Accesible: navegable con teclado, contraste correcto.
- Anuncios que no tapen la interfaz ni interrumpan la edición. Página de privacidad explicando que la partida no sale del dispositivo.
- Landing con SEO por juego ("editor partida Pokémon Blanco", etc.).

7. Riesgos

Riesgo	Mitigación
PKHeX.Core no funciona bien en WASM o pesa demasiado	Es lo primero que se valida (Fase 0). Plan B: usar solo las partes necesarias o portar la lógica de una generación concreta.
Corromper partidas de usuarios	Copia del original siempre disponible, tests de ida y vuelta (cargar → exportar → comparar), avisos claros.
Reclamaciones de Nintendo / The Pokémon Company	No alojar ROMs ni assets oficiales; preferir iconos propios o genéricos; no usar "Pokémon" como nombre o logo del sitio. Los ingresos por anuncios no deberían ser la única fuente de ingresos.
Licencia GPL	Repositorio público desde el inicio con la licencia indicada y créditos a PKHeX.
Mantenimiento con juegos nuevos	Depender de las actualizaciones de PKHeX.Core; actualizar la dependencia periódicamente.

8. Instrucciones para Claude Code

- Empezar por la **Fase 0** y no pasar a la siguiente hasta que funcione y esté probada.
- Crear un `CLAUDE.md` con: estructura del proyecto, decisiones tomadas, cómo compilar, cómo ejecutar los tests.
- Commits pequeños y descriptivos; una rama por fase.
- Antes de implementar cada fase, proponer un plan breve y esperar confirmación.
- Tests automáticos de ida y vuelta para cada juego soportado.
- Preguntar antes de añadir dependencias nuevas o servicios externos.

9. Criterios de aceptación del MVP

1. Puedo subir un .sav de al menos un juego de cada generación compatible y la web detecta cuál es.
2. Veo mi equipo y mis cajas con los datos correctos.

3. Cambio el nivel de un Pokémon y el dinero, descargo la partida y funciona en el emulador o la consola.
4. Todo funciona en el móvil y sin que la partida salga del navegador.